
WHITE PAPER · AI OPERATING GOVERNANCE

The Demo-to-Production Gap

A good demo is not proof for real work

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

A background image showing three people (two men and one woman) sitting around a table in a meeting. They are looking at laptops and talking. The image is dimmed and serves as a background for the text.

EXECUTIVE SUMMARY

Approve an AI tool for real work only on proof from real work, and only for the situations that proof covers.

A demo shows that an AI tool can produce a good result when someone picks the inputs and runs it. Real work brings ordinary users, messy cases, and pressure, and a demo says nothing about those. Seven questions settle whether a tool is ready: does the whole job improve, who owns the outcome, how does the tool handle difficult cases, what happens when it fails, can its safeguards be shown to work, who funds support after launch, and who can shut it off. The owner of the business process the tool supports records the answers in a one-page written approval that states what the tool may be used for, who is accountable, and what stops it, and revisits it when the tool, the vendor, or the workload changes.

- 01** A pilot run by the team that built the tool, with hand-picked cases and eager users, is just a longer demo. Only real users working on real cases show how the tool performs in ordinary work.
- 02** A weak answer to one of the seven questions does not have to stop the project. Approve the tool only for the cases the evidence covers, and widen the approval as the evidence grows.
- 03** At approval, put the costs the demo never showed into the budget, next to the benefits: handling the cases the tool gets wrong, supporting it after launch, and shutting it off if it fails. Funded up front, they stay visible instead of becoming surprises.

CONTENTS

The argument	Lifecycle support
What a demo actually proves	Stop authority
The seven production questions	Putting the questions together
Workflow fit	Replacing the demo approval
Operating ownership	What the record shows over time
Representative cases	The strongest counterargument
Exception recovery	What counts as proof
Observable controls	

The argument

A demo proves one thing: an AI tool, whether bought or built in-house, can produce a good result when someone has picked the inputs and the setting. Real work needs more. It needs proof that the business can count on the tool every day, with ordinary users and messy inputs. A demo and an enthusiastic pilot often show more about how much the team wants the tool to work than about whether it will work for everyone else. The proof that closes the gap is named owners, tested hard cases, a plan for failures, safeguards you can check, funded support, and a way to shut the tool off.

I have sat through plenty of demos built to sell a room. Clean inputs go in, an impressive result comes out, and the room is sold in minutes. What the room never sees is how the inputs were chosen: a handful of cases picked from thousands, with none of the messy ones that make up a third of the real work. The gap between those cases and the company's actual workload is the entire operating problem, and the demo left it out before anyone entered the room.

There is nothing dishonest about this. Demos exist to show what is possible, and hand-picked inputs, skilled presenters, and rehearsed steps are how they work. The governance failure comes afterward, when a good demo is quietly treated as proof the tool will work in daily use. That is when a tool that shone under ideal circumstances is approved for ordinary users, ordinary data, and ordinary pressure, with no one asking what changed.

Large surveys of business leaders keep finding the same result: plenty of experimentation with AI, but far less business value.[1][2] Closing that gap takes different evidence, not better demos or bigger pilots. This paper lays that evidence out as seven production questions, each answered before real work depends on the tool. They come from my own portfolio and AI-governance work, and the paper is written for AI investment owners, product and technology leaders, workflow owners, and risk executives.

What a demo actually proves

A demo is a proof of concept: under some circumstances, this tool produced this result. Taken at face value, that shows only that a good result is possible, not that it is

dependable. A proof of concept is not a finished product. It cannot show what makes a workflow ready for production: how the tool behaves across all the real inputs, how it performs with users who did not build it, and what happens when something it depends on goes down. It also cannot show who will carry the cost of owning the tool after the launch team moves on, whether the operations team, the vendor, or the business. That ongoing cost is where AI tools built on machine learning in particular pile up what engineers call technical debt: shortcuts and hidden dependencies that cost more to maintain the longer they sit. The debt hides in the data the tool depends on, how it is configured, and a changing outside world, and a demo run once by its builders never reveals it.[3]

Approving a tool for production claims that it works on an ordinary day, not just a good one. What stands between a demo and that claim is hand-picking: someone chose the inputs, the operators, and the users so the demo would look effortless.

Hand-picking goes beyond the inputs. The demo's operator knows which phrasings work. Pilot users were trained; later users will skim the instructions. The demo environment also leaves out the everyday obstacles of a real workplace: restrictions on who may see or change which records, data that is incomplete or out of date, and the end-of-quarter rush. Every obstacle the demo removes is like a small loan against production, and the loans come due together in the first month of ordinary use. That is why so many tools "worked in the pilot and struggled when rolled out widely." Real use exposed what the demo had avoided. The same story shows up in aggregate data. MIT NANDA's preliminary 2025 study, as reported by Fortune, found that only about 5 percent of AI pilot programs achieved rapid revenue acceleration, while the vast majority stalled with little to no measurable impact on P&L.[4]

Pilots are better than a proof of concept, and they carry the same weakness in a smaller form. Anyone who has watched software move from beta to release candidate to first release to service pack knows the gap: each stage catches problems the one before it missed. A pilot run by the build team, with chosen cases and enthusiastic users, is a longer demo. It produces real usage data, which makes it feel like production evidence, but the weakness is built in: the volunteers chose to take part, the hard cases were put off until later, and the engineers watching the queue handled by hand the cases the tool could not. A pilot often misses the unusual cases that ordinary work produces, so it still needs substantial testing and investment before rollout. Reaching production is also not the same as delivering value. A McKinsey-Oxford study of more than 5,400 IT projects found that large IT projects ran 45 percent over budget on average and delivered 56 percent less value than predicted.[5]

Demos are fine. The mistake is using one to answer the wrong question. A demo answers “is this possible?” A pilot answers “can motivated people make this useful?” Approving production use needs a third question: “can the business depend on this in ordinary, day-to-day work?” That question has seven parts.

Why does a good demo get treated as proof so easily? Three forces, none of them technical. Momentum: by the time the demo lands, the budget and the executive sponsor already want a reason to go ahead, and the demo supplies one. Visibility: a demo’s success is vivid and shared in a room, while production failures are scattered, statistical, and in the future. Incentives: the champion who found the tool is rewarded for the win, while the risks and ongoing costs earn no one a reward. A production gate, meaning a formal checkpoint that decides whether a tool goes into real use, exists to give those risks and costs an advocate. By default they have none.

The seven production questions

Each question names a kind of evidence that a demo cannot supply. Together they form the gate between an experiment that worked and real work that depends on the tool. The answers go into a one-page written approval: a record of what the tool may be used for, who owns it, and what stops it. A weak answer to one question does not have to end the project. It limits the approval to the cases and uses the evidence actually covers.

Workflow fit

The first question asks whether the whole job got better, not just the one step the tool performs. Production evidence starts with the complete path, from start to finish. Where does the tool’s input come from, and is it clean, complete, and on time? Which decisions or next steps depend on what the tool produces? Who hands work to whom? What timing does the operation actually need? A tool can do its own task well while the rest of the process eats up the time saved: staff re-entering the output into another application, reformatting it to match company templates, and chasing approvals that the automation cannot request on its own.

The evidence is a map of the job from start to finish, with measurements where one step hands off to the next: how long the whole job takes, how many times each case is touched, how often work is redone, and how long cases wait between steps. The test is blunt. Did the complete job improve, or did one step get faster while the job

took just as long? Proceed only when the whole job improves, without pushing the burden onto someone else's team.

Here is a concrete example of the failure. A tool extracts renewal dates and obligations from contracts, and that step gets fast. The job itself is "obligations tracked and renewals acted on." Between the two sit scanning of incoming paperwork, a legal review that trusts nothing it has not seen, another application that needs the output retyped, and a renewals team working on a different calendar. Speeding up the extraction without touching those handoffs produces faster input to the same bottlenecks. An end-to-end measurement would show it. The demo's ninety seconds never will.

Operating ownership

The second question asks who answers for the consequences. The owner of the AI model is not the business owner of the process the tool supports. When the tool misses an obligation and a renewal lapses, the harm lands in that business process, and only its business owner can accept the risk, decide what the tool may and may not be used for, and decide what happens next. Pilots run fine without this clarity because the build team fills every gap by hand. Production cannot.

The evidence is named acceptance: a business owner who signed off on what the tool may be used for, technical and data owners with agreed service expectations, a risk owner for the safeguards, and a support team with real capacity. The key document is the business owner's written approval, covering what the workflow may depend on the tool for, what stays with people, and what evidence would change either answer. Hold production use until the business team receiving the tool owns both the benefit and the response to failures. A tool whose harms have no owner is running in the business with no one in charge.

Ownership is also where vendor relationships get honest. A vendor demo shows the vendor's confidence; a written approval requires the buyer's. The business owner who has to sign has questions procurement never asked: what happens to our cases when your model updates, how often does it fail on inputs like ours, who do we call at 2 a.m.? Those questions, asked before signature, are the cheapest due diligence available, and how comfortable the vendor is answering them is itself evidence.

Representative cases

The third question goes after the demo's biggest shortcut: the choice of inputs. Accuracy on easy cases says little about the rare, unclear, out-of-date, and half-complete inputs that take up most of the judgment in daily operations. Build the test set from real history, plus deliberately hard cases, the ones built to break the tool: documents with long chains of amendments, scanned pages, the cases users mishandle, and the inputs a rushed or careless user might actually submit.

The evidence is a test set with a documented origin: where each type of case came from, how often each occurs, results for each type, and an analysis of failures weighted by how much harm they cause, rather than a single accuracy number. Just as important is a clear statement of which cases the results actually cover. The written approval then follows naturally: permit use on the cases that were tested, exclude what was not, and widen the approval only as the tested cases widen. Untested means off-limits, not approved by silence.

Building the hard-case test set is faster than teams expect, because the people doing the work already know where things break. An afternoon with them produces the list: the formats that break things, the cases that get argued about, the inputs that arrive half-complete every quarter-end. Add a sample from real history with its natural mess left in, and the test set exists. What it costs is nerve: the willingness to score the tool on the cases the team was hoping to avoid.

Exception recovery

The fourth question assumes failure and asks what happens next. Some cases will fall outside what the tool can handle; these are exceptions. Production readiness covers five things: detection (the workflow finds out quickly that the tool could not handle a case), routing (the case reaches someone with context and authority), intervention (a person can act before harm spreads), rollback (partial actions can be undone), and a return to a known good state. A pilot celebrates average performance. An operation lives with its worst cases.

Only proof from actual operation counts here, not diagrams. How fast are failures detected? How long did recovery take in rollbacks that actually happened? How many cases are waiting for a person to handle them, and for how long? What does it cost to staff the exception process at real volume? That last number quietly decides the case. If exceptions use more specialist hours than the automation saves, the business case has flipped, however accurate the tool is.

Design the exception process as a workflow, not a bucket. Someone specific receives the case, with context attached: what came in, what the tool attempted, and why it stopped. A service expectation, such as how quickly the case must be handled, matches the consequence. The common mistake is a generic manual-review pile, which absorbs failures until an aging backlog becomes an incident of its own. If the pilot never fed real cases into this process, it has not been tested. It has only been drawn.

Observable controls

The fifth question turns rules into proof. A rule that sensitive data stays out of what people type into the tool, or that big actions need approval, is just a sentence until the tool can show when the rule was approached, broken, or enforced. Controls are the safeguards and checks that keep the tool inside what it is approved to do, and production use needs controls that leave evidence as they run: logs tied to users and cases, review records, alerts that fired and were handled, and regular tests that the control still works.

Insist on the difference between a control that exists and a control you can inspect. A control that merely exists passes a design review. Surviving an incident, an audit, or a customer's due-diligence checklist takes one you can inspect. The rule cuts both ways: limit what the tool is allowed to do to what its controls can demonstrate, and widen that authority only as visibility grows with it.

Visible controls also protect the tool's advocates. Some quarter, someone will question the tool, after an incident, during an audit, or when a new executive arrives. Whether its owners can produce the operating record decides whether the tool is defended or doomed. Tools with inspectable histories survive scrutiny; assurances alone do not, however well the tool was actually behaving.

Lifecycle support

The sixth question prices the years after launch. Vendors update their models on their own schedules. The instructions and reference documents the tool draws on drift. Policies change. Users come and go. A demo has no lifecycle, and a pilot's lifecycle is the engineers' attention. Production needs funded, owned, standing capacity: keeping the tests current, responding to vendor changes, supporting users, handling incidents, and retesting regularly.

The evidence is a support plan with numbers in it: expected ticket volume, how often the tool is retested and by whom, how vendor change notices get handled, budgeted hours, and named successors for the knowledge that lives in the build team's heads. A launch team that disbands after go-live is the most reliable predictor of quiet decline a year later. Do not roll out a tool whose continuing costs are unfunded. The work does not disappear. It lands on whoever notices last.

Budget the lifecycle at approval time, in the same document as the benefit case, because that is the only moment the numbers compete fairly. A tool that saves four thousand hours a year and needs twelve hundred in standing support is still worth running. The twelve hundred should still be visible while the four thousand is being celebrated, not discovered next year as an unfunded surprise.

Stop authority

The seventh question is the one companies most reliably skip: who can turn it off, on what evidence, with what fallback (the manual procedure that takes over)? Without a stop agreed in advance, weak performance continues by default. Every trigger level gets debated after the fact. Every proposal to pause fights adoption momentum and sunk cost. Continuing becomes the path of least resistance, whatever the numbers say.

The evidence is a stop plan that has been rehearsed: named stop conditions with measurable trigger levels, a responsible owner, a way to tell everyone affected, a fallback that has actually run (the manual procedure still works and the team still knows it), and restart criteria that say what evidence restores service. The rule: stop or narrow use when a stop condition is met, regardless of momentum. An untested fallback is a hope. A tested one is why stopping is politically possible at all.

Stop authority changes behavior even when it is never used. A build team that knows the tool can be paused designs for it: cleaner records of what the tool did, better logging, actions that can be reversed. An operating team that trusts the stop mechanism reports problems earlier, because raising a hand no longer means owning a crisis alone.

Putting the questions together

Together the seven questions replace the demo's built-in question, "is this impressive?", with the operating one: what may this workflow depend on the tool for,

within what scope, owned by whom, watched how, supported by what, and stoppable when? The gate's output is a written approval with those blanks filled in, not a plain yes or no. Written that way, the decision can be inspected, revisited, and sized to the evidence: a narrow approval for thin evidence, a wider one as evidence accumulates.

EXHIBIT 1

The production readiness path

- 1 **Workflow fit**
Did the complete job improve, or just the part shown in the demo?
- 2 **Operating ownership**
Who answers for the consequence when the output is wrong?
- 3 **Representative cases**
Was it tested on real cases, including the hardest ones?
- 4 **Exception recovery**
When the tool cannot handle a case, what happens next?
- 5 **Observable controls**
Can the tool show that its safeguards were enforced?
- 6 **Lifecycle support**
Who funds and owns the years after launch?
- 7 **Stop authority**
Who can turn it off, and with what tested fallback?

The questions also depend on each other. Representative cases feed the design of the exception process. Exception costs feed support budgeting. Observable controls make stop conditions enforceable. Ownership makes every other answer accountable. In practice the gate fails at its weakest answer, which is why partial credit is dangerous: six strong answers and no stop authority is a tool that will run until its first bad month becomes a bad quarter.

Replacing the demo approval

The gate installs as a replacement for the demo-driven decision, not an addition to it:

- Replace the demo approval with a plan to gather proof from real work.
- Run the pilot inside the real workflow, with representative users and cases.

- Fund exception handling, testing, support, and incident response as ongoing work.
- Require a written approval that states permitted uses, exclusions, controls, owner, and stop conditions.

The written approval deserves a standard format, and it fits on one page. It is the business owner's record of the decision: written once at approval and revisited when the evidence, the tool, the vendor, or the workload changes. It is neither a form for the executive to fill in nor a worksheet to maintain. It states what uses are permitted and which tested cases they rest on; what is excluded; who owns what; which controls exist and where their evidence lives; the support plan and its budget line; the stop conditions and the fallback; and what evidence would widen or narrow any item. One page works precisely because the seven questions were answered before the page was written.

Running the pilot inside the real workflow is the highest-leverage change. It turns the pilot from a longer demo into a source of real evidence. Real inputs arrive in their real condition. Real users produce the unusual cases. Reality, not test design, exercises the exception process. If the pilot cannot run inside the real workflow for risk reasons, that fact is itself evidence of how much the tool has earned so far.

The gate needs an owner with standing, usually the team that runs the company's AI program, together with the risk team. It also needs a service promise: evidence plans are reviewed within a stated window, and small, reversible requests ride a fast track. The gate's authority should come from the approvals it enables, not from the approvals it withholds.

Expect the gate to slow enthusiasm down, and treat that as a benefit. Demo-driven decisions promote tools at the speed of impressions. The gate promotes them at the speed of evidence. The projects that survive are slower and more durable. The ones that die at the gate were going to die in production, at several times the price.

What the record shows over time

Across a portfolio of AI investments, the gate leaves measurable traces:

- Written approvals on file, with scope, owners, and stop conditions stated.
- Tools rolled out only to the cases that were tested, and approved uses widened only when new evidence supports it.

- Exception-handling capacity measured against automation savings before each expansion.
- Fallbacks rehearsed on schedule, and stop conditions that triggered real pauses.
- Support effort after launch compared with the funded plan.
- Production incidents traceable to untested kinds of cases, the number the gate exists to drive down.

The measures also keep the gate honest. A gate whose review time climbs while it catches no more problems is drifting toward ceremony. A portfolio whose approved uses have stopped widening despite accumulating evidence is being governed by caution instead of proof. Both show up in the record, and both can be corrected.

The last measure is the outcome. Incidents from untested kinds of cases are the signature of demo-driven rollouts, and their decline is the gate working. The first measure, written approvals on file, is the cheapest and most diagnostic: a company that cannot produce the document has not made the decision, whatever its tools are doing in production.

The strongest counterargument

Not every experiment needs production-grade controls. Exploring can be fast and loose when the output cannot cause real harm. The error is letting experiment results approve real use without passing through a deliberate gate.

EXHIBIT 2

Keep the gate honest

Match effort to risk

Review depth follows risk. Small, reversible requests clear within a week.

Review time

Track how long the gate takes as seriously as what it catches.

Widening approvals

Widen approvals as new evidence arrives, no sooner and no later.

The outcome measure

Incidents from untested kinds of cases, trending down.

A second objection carries real weight: seven questions can turn into seven committees, and a gate this thorough can harden into bureaucracy that teaches teams to route around it. The defense is matching depth to stakes and keeping to a schedule. The gate's depth should scale with the consequences: reversible, low-stakes automations should clear it within a week on thin evidence. The gate owner should also track how long the gate takes as seriously as what it catches. A gate that cannot say "small yes, quickly" will eventually be unable to say anything at all, because nothing will be brought to it.

What counts as proof

The whole argument comes down to what counts as proof. A demo proves a good result is possible; approval for real use has to be earned by evidence from real operation. The complete workflow, measured. Ownership, accepted. Hard cases, tested. Exceptions, rehearsed. Controls, visible. Support, funded. A stop that works. Between those two standards sits most of the distance between AI activity and AI value, and unlike how good the AI model is, that distance is entirely within a company's control.

For leaders, the working rule fits in a sentence: treat every impressive demo as the first step of an evidence plan. The enthusiasm is fine. It funds the work. The gate just makes sure enthusiasm and evidence arrive at the approval in the right order.

The demos will keep improving. Improving the demos is the vendors' job; building the gate is the company's, and the companies that build it are the ones whose impressive demos grow into tools their operations can actually depend on.

Notes and sources

[1] Alex Singla, Alexander Sukharevsky, Bryce Hall, Lareina Yee, and Michael Chui, with Tara Balakrishnan, "The state of AI in 2025: Agents, innovation, and transformation," McKinsey & Company, November 5, 2025.

<https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-2025>

[2] Deloitte AI Institute, "The State of AI in the Enterprise," 2026.

<https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-ai-in-the-enterprise.html>

[3] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," Advances in Neural Information Processing Systems 28 (NeurIPS 2015).

<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

[4] Aditya Challapally, Chris Pease, Ramesh Raskar, and Pradyumna Chari, "The GenAI Divide: State of AI in Business 2025," MIT NANDA, July 2025 (preliminary findings), as reported by Sheryl Estrada, "MIT report: 95% of generative AI pilots at companies are failing," Fortune, August 18, 2025.

<https://fortune.com/2025/08/18/mit-report-95-percent-generative-ai-pilots-at-companies-failing-cfo/>

[5] Michael Bloch, Sven Blumberg, and Jürgen Laartz, "Delivering large-scale IT projects on time, on budget, and on value," McKinsey & Company, 2012.

<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/delivering-large-scale-it-projects-on-time-on-budget-and-on-value>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.